Issue #55: Email Channel Integration — Implementation Design

Table of Contents

Overview . . . 2
System Architecture . . . 2
Inbound Email Flow . . . 3
- New Ticket (current behavior) . . . 3
- Reply to Existing Ticket (new behavior) . . . 3
- Email-to-Project Routing (existing) . . . 3
- Priority Detection (existing) . . . 3

Outbound Email Flow . . . 4
- Agent Reply → Email to Customer . . . 4
- Status Change → Notification Email . . . 4
- Auto-Reply on Ticket Creation . . . 4

Email Threading Strategy . . . 5
New Files . . . 5
- `src/lib/email.ts` — SMTP Service . . . 5
- `src/lib/email-templates.ts` — HTML Templates . . . 6
- `src/lib/email-parser.ts` — Thread Detection & Quote Stripping . . . 6

Modified Files . . . 6
- `src/types/index.ts` . . . 6
- `src/app/api/email/webhook/route.ts` . . . 7
- `src/app/api/devops/tickets/[id]/comments/route.ts` . . . 8
- `src/app/api/devops/tickets/[id]/state/route.ts` . . . 8

Dependencies . . . 8
Environment Variables . . . 9
Phased Implementation . . . 9
- Phase 1: Outbound Email + Auto-Reply . . . 9
- Phase 2: Inbound Thread Detection . . . 9
- Phase 3: Status Change Notifications . . . 9
- Phase 4: CC/BCC + Attachments (Future) . . . 10

Key Design Decisions . . . 10
Verification Plan . . . 10
Security Considerations . . . 11

Overview

This document describes the full email channel integration for ZapDesk. The goal is bidirectional email: customers email in to create tickets, and agents reply via the ZapDesk UI which sends emails back to the customer.

Current state: The inbound webhook (`POST /api/email/webhook`) already creates tickets from emails using PAT authentication and domain-based project routing. However, there is no outbound email, no thread detection for replies, and no status change notifications.

Target state: Full bidirectional email with threading, auto-replies, agent reply forwarding, and status notifications.

System Architecture

```
┌──────────────────────┐      ┌──────────────────────────────┐
┌──────────────────────────────────────────────────────┐
│  Customer  │──────>│  Exchange      │──────>│  Azure Logic App          │
│  Mailbox   │<──────│  Online        │     │  (Office 365 connector)    │
└──────────────────────┘      └──────────────────────────────┘
└──────────────────────┬──────────────────────────────────────┘
    ▯                                          │
     │                                          ▯
     │
┌──────────────────────────────────────────────────────────────────────────────────┐
     │                │          ZapDesk (Next.js)              │
     │                │                                         │
     │                │   POST /api/email/webhook               │
     │                │   ▯ Validate webhook secret             │
     │                │   ▯ Detect thread (reply vs new)        │
     │                │   ▯ Route to project by domain          │
     │                │   ▯ Create ticket OR add comment        │
     │                │   ▯ Send auto-reply confirmation        │
     │                │                                         │
     │                │   Agent UI (TicketDetail)               │
     │                │   ▯ Add comment → email customer        │
     │                │   ▯ Change status → notify customer     │
     │                │                                         │
     │                │   Email Service (src/lib/email.ts)      │
     │                │   ▯ SMTP transport (nodemailer)         │
     │                │   ▯ HTML templates                      │
     │                │   ▯ Threading headers                   │
     │                │   ▯ Fire-and-forget sends               │
     │
└──────────────────────────────┬──────────────────────────────────────────────────┘
     │                                 │
     │                                 ▯
     │                ┌──────────────────────────────────────────────────────┐
```

```
        └──────────────────────────────┤ SMTP (Office 365)        │
                                       └──────────────────────────────────────────┘
```

Inbound Email Flow

New Ticket (current behavior)

1. Customer sends email to configured support mailbox
2. Azure Logic App extracts fields and POSTs to `/api/email/webhook`
3. Webhook validates `x-webhook-secret` header
4. Sender domain mapped to Azure DevOps project via `getProjectFromEmail()`
5. Work item created with tags `ticket; email` using PAT auth
6. **NEW:** Auto-reply sent to customer with ticket number

Reply to Existing Ticket (new behavior)

1. Customer replies to a ZapDesk email (subject contains `[ZapDesk #123]`)
2. Azure Logic App forwards reply to webhook
3. Webhook detects `[ZapDesk #<id>]` in subject line
4. Quoted text and email signature stripped from body
5. Clean reply body added as comment to work item `#123`
6. Confirmation email sent to customer

Email-to-Project Routing (existing)

```
sender@cairnhomes.com
  → Extract domain: cairnhomes.com
  → Check cache (5-min TTL)
  → Query DevOps project descriptions for "Email: cairnhomes.com"
  → Return project name
  → Fallback: hardcoded domain map
```

Priority Detection (existing)

Subject keyword	Priority
`urgent`, `critical`, `emergency`	1 (Urgent)
`high priority`, `important`	2 (High)
`low priority`, `when you can`	4 (Low)

Subject keyword	Priority
(default)	3 (Normal)

Outbound Email Flow

Agent Reply → Email to Customer

1. Agent types reply in ZapDesk UI
2. `POST /api/devops/tickets/[id]/comments` adds comment to DevOps work item
3. **NEW:** If ticket has `email` tag AND SMTP is configured:
   - Fetch requester email from work item `CreatedBy` field
   - Render reply HTML template
   - Set threading headers (`In-Reply-To`, `References`, `Message-ID`)
   - Set subject: `Re: [ZapDesk #123] Original Subject`
   - Send via nodemailer (fire-and-forget — failure must never block the comment)
4. If ticket does NOT have `email` tag → no email sent (UI-only ticket)

Status Change → Notification Email

1. Agent changes ticket status via `PATCH /api/devops/tickets/[id]/state`
2. State updated in DevOps
3. **NEW:** If ticket has `email` tag AND SMTP is configured:
   - Render status notification template: "Your ticket #123 is now Resolved"
   - Send notification email to requester

Auto-Reply on Ticket Creation

1. Webhook creates ticket `#456`
2. **NEW:** Send confirmation email to sender:

```
To: customer@example.com
Subject: [ZapDesk #456] Re: Original Subject

Hi,

We've received your request and created ticket #456.
Our team will review it shortly.
Reply to this email to add details.

— ZapDesk Support
```

Email Threading Strategy

All threading is based on the subject prefix `[ZapDesk #<workItemId>]`.

```
1. Customer → ZapDesk
   Subject: "Login issue"
   → Creates ticket #123

2. ZapDesk → Customer (auto-reply)
   Subject: "[ZapDesk #123] Re: Login issue"
   Message-ID: <zapdesk-123-1709734800@domain>

3. Agent → Customer (reply)
   Subject: "Re: [ZapDesk #123] Re: Login issue"
   In-Reply-To: <zapdesk-123-1709734800@domain>
   Message-ID: <zapdesk-123-1709738400@domain>

4. Customer → ZapDesk (reply)
   Subject: "Re: [ZapDesk #123] Re: Login issue"
   → Webhook detects [ZapDesk #123]
   → Adds as comment to ticket #123 (not new ticket)
```

Message-ID format: `<zapdesk-{workItemId}-{unixTimestamp}@{smtpFromDomain}>` — deterministic, no storage needed.

New Files

`src/lib/email.ts` — SMTP Service

Responsibilities:

- Create nodemailer SMTP transport from env vars (`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`)
- `isSmtpConfigured()` — returns `true` only if all required SMTP vars are set
- `sendTicketConfirmation(to, ticketId, originalSubject)` — auto-reply on ticket creation
- `sendAgentReply(to, ticketId, subject, agentComment)` — forward agent reply to customer
- `sendStatusNotification(to, ticketId, subject, newStatus)` — notify on status change
- All send functions are fire-and-forget: catch errors, log them, never throw

Transport config:

```
{
  host: process.env.SMTP_HOST,
  port: Number(process.env.SMTP_PORT) || 587,
```

```
  secure: false, // STARTTLS
  auth: {
    user: process.env.SMTP_USER,
    pass: process.env.SMTP_PASSWORD,
  },
}
```

`src/lib/email-templates.ts` — HTML Templates

Plain TypeScript functions returning HTML strings. No template engine dependency.

Templates needed:

- `ticketConfirmationTemplate(ticketId, originalSubject)` — "We received your request"
- `agentReplyTemplate(ticketId, agentName, replyHtml)` — agent's comment as HTML email
- `statusUpdateTemplate(ticketId, subject, oldStatus, newStatus)` — "Your ticket is now Resolved"

Each template includes:

- ZapDesk branding header
- Ticket reference number
- Footer: "Reply to this email to add details to your ticket"
- Inline CSS (no external stylesheets for email compatibility)

`src/lib/email-parser.ts` — Thread Detection & Quote Stripping

- `extractTicketIdFromSubject(subject: string): number | null` — regex for `[ZapDesk #(\d+)]`
- `stripQuotedText(body: string): string` — remove `>` quoted lines, `On … wrote:` blocks, and common signature markers (`--`, `_`)
- `stripHtmlQuotedText(html: string): string` — remove `<blockquote>` and `.gmail_quote` divs

Modified Files

`src/types/index.ts`

Add to `EmailWebhookPayload`:

```
export interface EmailWebhookPayload {
  from: string;
  to: string;
  subject: string;
  body: string;
```

```
  cc?: string;       // NEW: CC recipients
  replyTo?: string;  // NEW: Reply-To header
  attachments?: Array<{
    filename: string;
    contentType: string;
    content: string; // base64 encoded
  }>;
}
```

src/app/api/email/webhook/route.ts

Changes:

1. **Thread detection:** Before creating a new ticket, check if subject contains `[ZapDesk #<id>]`
   - If YES → extract ticket ID, strip quoted text, add as comment to existing work item
   - If NO → create new ticket (existing behavior)
2. **Auto-reply:** After creating a new ticket, call `sendTicketConfirmation()` (fire-and-forget)
3. **Reply confirmation:** After adding comment to existing ticket, optionally send "reply received" confirmation

Pseudocode for the updated flow:

```
// 1. Validate webhook secret (existing)
// 2. Parse payload (existing)
// 3. Extract sender email (existing)

// 4. NEW: Check for thread
const ticketId = extractTicketIdFromSubject(subject);

if (ticketId) {
  // Reply to existing ticket
  const cleanBody = stripQuotedText(body);
  await patService.addCommentToWorkItem(ticketId, cleanBody, senderEmail);
  sendTicketReplyConfirmation(senderEmail, ticketId, subject); // fire-and-forget
  return { success: true, ticketId, action: 'comment_added' };
}

// 5. Route to project (existing)
// 6. Create ticket (existing)

// 7. NEW: Send auto-reply
sendTicketConfirmation(senderEmail, workItem.id, subject); // fire-and-forget

return { success: true, ticketId: workItem.id, action: 'ticket_created' };
```

src/app/api/devops/tickets/[id]/comments/route.ts

After successfully adding the comment to DevOps:

```
// Existing: addComment() succeeds

// NEW: Send email if ticket was created via email
const tags = workItem.fields?.['System.Tags'] || '';
if (tags.includes('email') && isSmtpConfigured()) {
  const requesterEmail = workItem.fields?.['System.CreatedBy']?.uniqueName;
  const subject = workItem.fields?.['System.Title'];
  if (requesterEmail) {
    sendAgentReply(requesterEmail, ticketId, subject, comment); // fire-and-forget
  }
}
```

> The `workItem` is already fetched in the project search loop. The email send is wrapped in try/catch and must never cause the comment API to fail.

src/app/api/devops/tickets/[id]/state/route.ts

After successfully updating the state:

```
// Existing: updateTicketState() succeeds

// NEW: Send notification if ticket was created via email
const tags = workItem.fields?.['System.Tags'] || '';
if (tags.includes('email') && isSmtpConfigured()) {
  const requesterEmail = workItem.fields?.['System.CreatedBy']?.uniqueName;
  const subject = workItem.fields?.['System.Title'];
  if (requesterEmail) {
    sendStatusNotification(requesterEmail, ticketId, subject, state); // fire-and-
forget
  }
}
```

Dependencies

```
bun add nodemailer
bun add -d @types/nodemailer
```

No other new dependencies. Templates are plain TypeScript functions.

Environment Variables

Variable	Purpose	Required
`EMAIL_WEBHOOK_SECRET`	Authenticates inbound webhook requests	For inbound (existing)
`AZURE_DEVOPS_PAT`	Service account for webhook ticket creation	For inbound (existing)
`SMTP_HOST`	SMTP server (e.g., `smtp.office365.com`)	For outbound
`SMTP_PORT`	SMTP port (e.g., `587`)	For outbound
`SMTP_USER`	SMTP username / email address	For outbound
`SMTP_PASSWORD`	SMTP password or app password	For outbound
`SMTP_FROM`	Sender email address	For outbound
`SMTP_FROM_NAME`	Sender display name (e.g., "ZapDesk Support")	For outbound

All SMTP variables are already documented in `.env.example`. No new env vars needed.

Phased Implementation

Phase 1: Outbound Email + Auto-Reply

- Add `nodemailer` dependency
- Create `src/lib/email.ts` — SMTP transport, send functions, `isSmtpConfigured()`
- Create `src/lib/email-templates.ts` — confirmation, reply, and status templates
- Modify `src/app/api/email/webhook/route.ts` — send auto-reply after ticket creation
- Modify `src/app/api/devops/tickets/[id]/comments/route.ts` — email requester on agent reply

Value: Customers get notified when agents respond.

Phase 2: Inbound Thread Detection

- Create `src/lib/email-parser.ts` — `extractTicketIdFromSubject()`, `stripQuotedText()`
- Modify webhook to detect `[ZapDesk #ID]` and add as comment instead of creating new ticket
- Need to add `addComment()` capability to `AzureDevOpsServiceWithPAT` class

Value: Customer replies stay in the same ticket thread.

Phase 3: Status Change Notifications

- Modify `src/app/api/devops/tickets/[id]/state/route.ts` — send notification on state change
- Add status update template to `email-templates.ts`

Value: Customers know when their ticket progresses.

Phase 4: CC/BCC + Attachments (Future)

- Process inbound email attachments → upload to DevOps via `addAttachmentToWorkItem()`
- Parse CC recipients, include on outbound replies
- Extend `EmailWebhookPayload` type with `cc`, `replyTo` fields

Value: Full email conversation fidelity.

Key Design Decisions

Decision	Choice	Rationale
Transport	nodemailer + SMTP	Env vars already defined in `.env.example`, battle-tested library
Templates	Plain TS functions	No extra dependency; email templates are simple HTML
Threading	Subject prefix `[ZapDesk #ID]`	Works across all email clients, no custom DevOps fields needed
Message-ID	`<zapdesk-{id}-{ts}@domain>`	Deterministic, no storage needed
Metadata	Work item tags + description	No custom DevOps fields needed, works across process templates
Failure mode	Fire-and-forget	Email failure must never block ticket/comment operations
Internal notes	Skip email	Comments prefixed with `[Internal Note]` should not be emailed to customers

Verification Plan

1. `bun add nodemailer && bun add -d @types/nodemailer`
2. Configure SMTP + webhook env vars in `.env.local`
3. **Inbound test:** `curl -X POST localhost:3000/api/email/webhook` with sample payload → ticket created
4. **Auto-reply test:** Check customer inbox for confirmation email with `[ZapDesk #ID]` subject
5. **Outbound test:** Add comment to email-tagged ticket → customer receives agent reply email
6. **Thread test:** Reply to notification email → webhook adds as comment (not new ticket)
7. **Non-email test:** Comment on UI-created ticket (no `email` tag) → no email sent
8. **Internal note test:** Add internal note to email ticket → no email sent
9. **Failure test:** Misconfigure SMTP → comment still succeeds, error logged
10. `bun run check` passes

Security Considerations

- Webhook authenticated via shared secret (`x-webhook-secret` header)
- Sender domain validated against project mappings
- Internal notes (`[Internal Note]` prefix) must never be emailed to customers
- Email addresses must not appear in public repository code or docs (use generic references)
- SMTP credentials stored in environment variables, never in code
- TLS enforced for SMTP connections (STARTTLS on port 587)
- Rate limiting recommended at infrastructure level (Logic App / API gateway)